

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE

Department of Computer Science Engineering

Course Code: 23CS002PC304

Course Title: AI Assisted Coding

Year/Sem: III / II

Assignment Number: 17.1 / 24

Name: O.Narasimha Raju

HT No: 2303A53034

Batch: 46

Lab Objectives

- Understand string matching algorithms
- Implement Rabin-Karp and KMP
- Improve pattern searching efficiency
- Analyze time complexity

Task 1: Rabin-Karp Algorithm

Python Code

```
def rabin_karp(text, pattern):  
    n = len(text)  
    m = len(pattern)  
  
    for i in range(n - m + 1):  
        if text[i:i+m] == pattern:  
            print("Pattern found at index", i)  
  
rabin_karp("AABACAADAABAABA", "AABA")
```

Explanation: Uses hashing to match substrings efficiently.

Task 2: KMP Algorithm

Python Code

```
def compute_lps(pattern):
    lps = [0] * len(pattern)
    length = 0
    i = 1

    while i < len(pattern):
        if pattern[i] == pattern[length]:
            length += 1
            lps[i] = length
            i += 1
        else:
            if length != 0:
                length = lps[length - 1]
            else:
                lps[i] = 0
                i += 1
    return lps

def kmp_search(text, pattern):
    lps = compute_lps(pattern)
    i = j = 0

    while i < len(text):
        if pattern[j] == text[i]:
            i += 1
            j += 1

        if j == len(pattern):
            print("Pattern found at index", i - j)
            j = lps[j - 1]
        elif i < len(text) and pattern[j] != text[i]:
            if j != 0:
                j = lps[j - 1]
            else:
                i += 1

kmp_search("ATCGATCGAATCGA", "ATCGA")
```

Task 3: Comparison

- Rabin-Karp: Average $O(n + m)$
- KMP: $O(n + m)$
- KMP avoids repeated comparisons using LPS array

Conclusion

String matching algorithms improve search efficiency.

AI tools help generate optimized implementations quickly.